Date: .... No: ....

Họ và Tên: Nguyễn Quân Vương
MSSV: N25DECE076
Lớp: E25CQCE02-N

Problem 1: 6.5
Problem 2: 2.5
Problem 3: 2.5
Problem 4: 0
6.5

Problem 1: Singly LinKed List - In-Place Reversal and Pointer tracing

Given a singly linKed list: head → [10] → [25] → [40] → [15] → NULL

- Task 1: Write a function/algorith in pseudo code to reverse the singly linKed list, including 3 pointer (prev, curr, next) to complete your function
- TasK 2: Draw a diagram or step-by-step pointer trace table showing the positions and values of the 3 pointers (prev, curr, next) after each iteration

Problem 2: Circular LinKed list - Insert at Head

Given a sing'r circular linKed list: head → [10] → [20] → [30] → (point back to head)

- TasK1: Write pseudo code for the function InsertHead (head, newValue) to insert a new node containing newValue at the front of the circular linKed list while maintaining its circular structure
- Task2: Trace the execution steps (in text de description or pointer diagrams) when inserting newValue = 5 into the given list

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

Problem 3: Doubly linked list - Insert before Target and Edge case

Given a doubly linked list containing integers

- Task 1: Write Pseudo code for InsertBefore(head, target, newValue) to insert a new node containing newValue immediately before the first occurrence of target
- Task 2: List the exact order of pointer assignments required and briefly justify why this specific sequence avoids memory leaks or broken links
- Task 3: Explicitly handle the following 2 edge cases in code/pseudo code
  1 Target is located at the head node (head)
  2 target does not exist in the list

Problem 4: Sorted Linked List - In order Insertion

Given a single linked list sorted in ascending order

head → [2] → [5] → [8] → NULL

- Task 1: Write the function SortedInsert(head, val) to insert val into the list such that the ascending order is preserve
- Task 2: construct a step-by-step trace table detailing pointer iteration and conclusion evaluations when inserting val = 6 into the list

Date: .... No: ....

(1,25) Problem 1

1,25 Task 1

```
void ReverseList(Node*& head) {
    Node* prev = NULL;
    Node* curr = head;
    Node* next = NULL;

    while (curr != NULL) {
        next = curr -> next;
        curr -> next = prev;
        prev = curr;
        curr = next;
    }
    head = prev;
    return head;
}
```

~~2,25~~ Problem 2

(2,25) ~~1,25~~ Task 1

1,0

```
void InsertHead(Node*& head, int newValue) {
    Node* newNode = new Node(newValue);

    if (head == NULL) {
        newNode -> next = newNode;
        head = newNode;
    }
    return head / return newNode;
}
```

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

```
Node* temp = head;
while (temp -> next != head)
    temp = temp -> next;
}

newNode -> next = head;
temp -> next = newNode;
head = newNode;
}
return head;
```

Task 2

head → [10] → [20] → [30] → (point back to head)

1 Initialization: A newNode is created with value 5. Its next is initially NULL. List is not empty, so we proceed

2 Traversal: Initialize temp = head (Point to [10])

- Step 1: temp → next is [20]. Move temp to [20]
- Step 2: temp → next is [30]. Move temp to [30]
- Step 3: temp → next is [10] (= head). Loop terminates. temp is now at the last node [30]

3 Pointer assignments:

- newNode → next = head; → [5] now points to [10].
- temp → next = newNode; → [30] now point to [5]
- head = newNode; → The head pointer now start at [5]

4 Structure Diagram:

head → [5] → [10] → [20] → [30]

2

No.

Date

2.5 Problem 3

2.5 Task 1; 3

```
void InsertBefore(DNode* &head, int target, int newValue
){
    Node* newNode = newNode(newValue)
    if(head == NULL) return; // Edge case Empty list
    {newNode->next = newNode; return newNode;
    // Task 3.1
    if (head->data == target) {
        DNode* newNode = new DNode(newValue);
        newNode->next = head;
        newNode->prev = NULL;
        head->prev = newNode;
        head = newNode;
        return;
    }

    DNode* curr = head;
    while (curr != NULL && curr->data != target
{
        curr = curr->next;
    }

    // Task 3.2
    if (curr == NULL) {
        return;
    }

    DNode* newNode = new DNode(newValue);
    newNode->next = curr;
    newNode->prev = curr->prev;
```

Thành Đạt

Scanned with CamScanner

curr → prev → next = newNode;
curr → prev = newNode;
}

(c) 15 pts x 2

Exact Order Required:

1 newNode → next = curr; (Connect new node foward to target)

2 newNode → prev = curr → prev; (connect new node backward to target's predecessor)

3 curr → prev → next = newNode; (Connect target's predeccessar foward to new node)

4 curr → prev = newNode; (connect target backward to new node)

Justification: Step 1 and 2 must be executed first. If step 3 or 4 are done too early, the curr → prev pointer is overwritten and permanently lost, causing broken list links and memory leaks.